

ADMINA-RH Deploiement & Configuration

Rapport technique complet des travaux de deploiement,
de configuration
et de resolution des problemes sur la plateforme
ADMINA-RH
hebergee sur Cloudflare Workers avec Supabase.

1. Contexte et Presentation du Projet	2
1.1 Environnement technique	2
2. Deploiement sur Cloudflare Workers	2
2.1 Configuration wrangler.toml	3
2.2 Probleme de taille du bundle (OOM)	3
3. Resolution des Erreurs de Connexion	3
3.1 Erreur "Profil introuvable"	4
3.2 Erreur "Invalid API key"	4
3.3 Erreur "Erreur de connexion : Veuillez reessayer"	4
4. Suppression des References Hotel/Cameroun	4
5. Analyse des Cles Supabase	5
6. Implementation du Login Fallback PostgREST	5
6.1 Architecture du fallback	5
6.2 Comptes de demonstration	6
7. Analyse et Memoire du Repository GitHub	6
8. Bilan Technique et Prochaines Etapes	7
8.1 Resultats obtenus	7
8.2 Prochaines etapes	7
8.3 Risques et recommandations	7

1. Contexte et Presentation du Projet

ADMINA-RH est une application web de gestion des ressources humaines concue comme un Systeme d'Information RH complet. Le projet a ete initialement developpe avec une orientation hoteliere (gestion de chambres, restauration, etc.) avant d'etre pivot vers un outil de GRH pure couvrant 31 domaines fonctionnels repartis sur 8 departements et 19 services. L'architecture technique repose sur Next.js 16 avec TypeScript, Tailwind CSS 4, shadcn/ui pour l'interface, et Supabase (Auth + PostgreSQL) pour le backend de donnees et l'authentification.

Le depot GitHub est heberge a l'adresse github.com/georgyfr/Admina_RH et contient l'ensemble du code source, les fichiers de configuration, ainsi que 13 documents PDF extraits dans le repertoire DEPARTEMENTS decrivant les specifications fonctionnelles de chaque departement. Le projet utilise une architecture modulaire avec plus de 100 pages React generees dynamiquement pour couvrir l'ensemble des 31 domaines fonctionnels, incluant le recrutement, la paie, les conges, les evaluations, la gestion des documents, et bien d'autres modules RH critiques.

1.1 Environnement technique

Technologie	Version / Detail	Role
Next.js	16.1.3 (Turbopack)	Framework frontend
TypeScript	5.x	Langage principal
Tailwind CSS	4.x	Styles utilitaires
shadcn/ui	Dernier	Composants UI
Supabase	PostgreSQL + GoTrue	Base de donnees + Auth
Cloudflare Workers	nodejs_compat	Hebergement serverless
Zustand	Dernier	Gestion d'etat client
Prisma	SQLite (local)	ORM local (supprime en prod)

Tableau 1 : Stack technique du projet ADMINA-RH

2. Deploiement sur Cloudflare Workers

Le deploiement d'ADMINA-RH sur Cloudflare Workers a necessite la mise en place d'un pipeline de build et de deploiement specifique. Cloudflare Workers impose une limite de taille de 3 MiB pour les scripts sur le plan gratuit, ce qui a constitue un defi technique majeur lors des premieres tentatives de deploiement. L'outil utilise est @opennextjs/cloudflare, un adaptateur qui convertit les applications Next.js en Workers Cloudflare via le format OpenNext.

Le processus de deploiement suit un workflow en deux etapes : d'abord, la commande 'npx opennextjs-cloudflare build' genere le bundle optimise dans le repertoire .open-next, puis la commande 'npx wrangler deploy' (ou 'npx opennextjs-cloudflare deploy') pousse le code vers l'edge de Cloudflare. Le deploiement est rapide, prenant environ 30 secondes pour un build complet et 8 secondes pour le transfert vers Cloudflare. L'application est accessible a l'URL <https://admina-rh.supdgeorgyfr.workers.dev>.

2.1 Configuration wrangler.toml

Le fichier wrangler.toml configure le Worker Cloudflare avec le compatibility_flag 'nodejs_compat' necessaire pour l'execution du code Next.js. Les variables d'environnement sont definies dans la section [vars] et incluent la URL Supabase, la cle anon publique, et la cle service_role secrete. Il est important de noter que les variables NEXT_PUBLIC_* sont inlined a la compilation par Next.js, tandis que les autres sont disponibles via l'objet env du Worker au runtime.

2.2 Probleme de taille du bundle (OOM)

Lors du premier deploiement reussi, le build a echoue avec une erreur 'Worker exceeded the size limit of 3 MiB'. La cause principale etait le moteur Prisma WASM (query_engine_bg.wasm) qui pesait 2.1 Mo seul, depassant la limite du plan gratuit. Le premier build a ete tue par le systeme (exit code 137 = OOM) en raison de la consommation memoire excessive de Turbopack.

La solution a consiste a remplacer completement Prisma par un stub (un objet vide qui leve des erreurs si appele en mode Supabase). Etant donne que l'application utilise exclusivement Supabase en production, le code Prisma n'etait jamais execute au runtime mais etait quand meme inclus dans le bundle par le bundler. Le remplacement a reduit la taille compressee de 2294 Ko a un niveau acceptable pour Cloudflare Workers.

Indicateur	Avant suppression	Apres suppression
Taille totale upload	> 3 MiB (rejete)	12 522 Ko (accepte)
Taille gzip	N/A (depasse)	2 294 Ko
Limite gratuite	3 MiB (3072 Ko)	Respectee
Fichier Prisma WASM	2 120 Ko	0 Ko (supprime)
Worker Startup	N/A	22 ms

Tableau 2 : Impact de la suppression de Prisma sur la taille du bundle

3. Resolution des Erreurs de Connexion

3.1 Erreur "Profil introuvable"

La premiere erreur critique rencontree etait le message 'Profil introuvable' lors de la tentative de connexion. Le probleme venait du fait que l'utilisateur existait dans Supabase Auth mais n'avait pas de ligne correspondante dans la table admina_users de la base de donnees PostgreSQL. Le login necessitait les deux : authentication via Supabase Auth (GoTrue) puis recuperation du profil depuis admina_users (PostgREST). La correction a ete apportee dans le fichier store.ts en ajoutant un mecanisme de fallback qui construit un profil a partir des metadonnees de l'utilisateur Auth si aucune ligne n'existe dans admina_users.

3.2 Erreur "Invalid API key"

La deuxieme erreur majeure etait 'Invalid API key' renvoyee par Supabase a chaque tentative d'appel API. L'analyse a revele que la cle anon publique utilisee avait une date d'expiration (exp: 1761972653, soit novembre 2025) qui etait deja depassee. De plus, les clees fournies initialement par l'utilisateur avaient un prefixe 'sb_publishable_*' et 'sb_secret_*' qui correspondent au format Stripe et non au format Supabase. Apres plusieurs echanges, l'utilisateur a fourni une nouvelle cle anon publique valide avec une expiration en 2036, resolvant definitivement ce probleme.

3.3 Erreur "Erreur de connexion : Veuillez reessayer"

Cette erreur etait causee par l'absence des variables NEXT_PUBLIC_SUPABASE_URL et NEXT_PUBLIC_SUPABASE_ANON_KEY dans le fichier .env avant la compilation. Les variables NEXT_PUBLIC_* sont inlinées par Next.js au moment du build et ne peuvent pas etre ajoutees a posteriori. La correction a consiste a ajouter ces variables dans le fichier .env avant de lancer le build. De plus, le wrangler.toml devait egalement etre mis a jour avec les clees pour qu'elles soient disponibles au runtime du Worker Cloudflare.

4. Suppression des References Hotel/Cameroun

Le projet original contenait de nombreuses references a un contexte hotelier et camerounais (noms d'hotels, ville de Douala, devise FCFA, secteur 'Hotellerie', etc.). Plus de 20 fichiers ont ete modifies systematiquement pour remplacer ces references par des valeurs generiques ou neutres. Le schema Prisma contenait egalement des valeurs par default liees a ce contexte (country: 'Cameroun', city: 'Douala', currency: 'XAF', sector: 'Hotellerie') qui ont ete remplacees par des tirets ('---') ou des valeurs neutres. La fonction de seed de donnees (seed.ts) contenait egalement des numeros de telephone camerounais et des adresses email specifiques qui ont ete neutralises.

Fichier modifie	Type de modification
prisma/schema.prisma	Valeurs par default neutralisees

src/lib/store.ts	defaultTenant neutralise
src/lib/datasource.ts	findTenantById neutralise
src/app/api/auth/login/route.ts	References supprimees
src/app/api/billing/route.ts	Prix et plans adaptes
src/lib/seed.ts	Donnees de demo neutralisees
20+ fichiers composants	Textes UI neutralises

Tableau 3 : Fichiers modifies pour la suppression des references hotel/cameroun

5. Analyse des Cles Supabase

L'analyse approfondie des cles Supabase a revele un comportement inattendu mais important a comprendre. La cle anon publique fonctionne correctement avec l'API PostgREST (donnees) et avec l'API GoTrue (signup), mais la cle service_role ne fonctionne qu'avec PostgREST. L'API GoTrue Admin rejette la cle service_role avec l'erreur 'invalid JWT: unable to parse or verify signature, token signature is invalid'. Cela signifie probablement que GoTrue et PostgREST utilisent des secrets JWT differents dans cette configuration Supabase.

Cle	PostgREST (donnees)	GoTrue (auth admin)
anon (publique)	Fonctionne	Fonctionne
service_role	Fonctionne	Rejetee (signature invalide)
sb_publishable_*	Non reconnu	Non reconnu (format Stripe)

Tableau 4 : Compatibilite des cles avec les API Supabase

6. Implementation du Login Fallback PostgREST

Face a l'impossibilite d'utiliser la cle service_role avec GoTrue (et donc de creer ou confirmer des utilisateurs dans Supabase Auth), une solution de contournement a ete implementee. Le login tente d'abord l'authentification Supabase standard (signInWithPassword), puis en cas d'echec, bascule vers un endpoint API dedie (/api/auth/login-direct) qui verifie directement l'utilisateur dans la table admina_users via PostgREST (qui fonctionne avec la cle service_role).

6.1 Architecture du fallback

Le fallback comprend trois nouveaux fichiers : un endpoint API (login-direct/route.ts), un endpoint de deconnexion (logout-direct/route.ts), et des modifications du middleware et du store Zustand. Le middleware a ete mis a jour pour accepter un cookie 'admina_auth' en plus de la session Supabase. Le store (store.ts) modifie loginWithPassword pour enchaîner les deux tentatives : Supabase Auth d'abord, puis le fallback PostgREST si la premiere echoue. Le logout efface a la fois la session Supabase et le cookie admina_auth.

6.2 Comptes de demonstration

Cinq comptes de demonstration sont disponibles dans la table admina_users, tous accessibles avec le mot de passe 'Admin@2024' en mode fallback. Ce sont les seuls comptes fonctionnels actuellement. Les autres utilisateurs doivent etre crees manuellement dans la table admina_users via Supabase. Il est important de noter que ce systeme de mot de passe unique est temporaire et doit etre remplace par une authentification robuste une fois la cle service_role corrige.

Email	Role	Statut
super.admin@admina-rh.demo	super_admin	Actif
admin@admina-rh.demo	tenant_admin	Actif
manager.admin@admina-rh.demo	manager	Actif
employe.demo@admina-rh.demo	employee	Actif
viewer.demo@admina-rh.demo	viewer	Actif

Tableau 5 : Comptes de demonstration disponibles

7. Analyse et Memoire du Repository GitHub

L'ensemble du repository GitHub a ete analyse et memorise. Le projet contient 31 domaines fonctionnels organises en 8 departements et 19 services. Chaque domaine dispose de ses propres pages, composants React, et endpoints API. Treize documents PDF ont ete extraits du repertoire DEPARTEMENTS, representant environ 640 000 caracteres et 290 pages de specifications fonctionnelles. Ces documents decrivent en detail les processus, les formulaires, et les regles metier de chaque departement RH.

La structure du repository comprend egalement les configurations de deploiement (wrangler.toml, .env), les scripts de construction (package.json, next.config), les styles globaux, et les composants d'interface reutilisables (boutons, inputs, cartes d'authentification, etc.). Cette memorisation complete permet de comprendre l'architecture globale et de naviguer rapidement dans le code pour apporter des modifications cibles.

8. Bilan Technique et Prochaines Etapes

8.1 Resultats obtenus

- Application deployee et accessible sur Cloudflare Workers
- Connexion fonctionnelle via fallback PostgREST direct
- Bundle optimise sous la limite de 3 MiB du plan gratuit
- References hotel/cameroun/FCFA supprimees de 20+ fichiers
- 5 comptes de demonstration operationnels
- Structure de 31 domaines fonctionnels memorisee
- Middleware et auth multicanal operationnels

8.2 Prochaines etapes

Plusieurs taches restent en attente pour finaliser le projet. La plus critique est la resolution du probleme de la cle service_role qui ne fonctionne pas avec l'API GoTrue Admin. Sans cette cle, il est impossible de creer des utilisateurs authentifies dans Supabase Auth de maniere programmatique, ce qui signifie que tout nouvel utilisateur doit etre cree manuellement via le Dashboard Supabase ou via la page d'inscription (avec confirmation email). Il est egalement necessaire d'executer les fichiers d'audit SQL presents dans le repertoire 'prompt-et-correction/Audit base de donnee' du repository GitHub, qui n'ont pas encore ete traites a ce jour. Enfin, la securite du login doit etre renforcee avec un systeme de mots de passe individuels et hashes au lieu du mot de passe unique actuel.

8.3 Risques et recommandations

Le principal risque technique reside dans la dependance au plan gratuit de Cloudflare Workers (limite de 3 MiB). Si l'application continue de croitre avec l'ajout de nouveaux domaines et services, la taille du bundle pourrait a nouveau depasser cette limite. Il est recommande de migrer vers un plan payant Cloudflare (5\$/mois) qui offre une limite de 10 MiB, ou d'optimiser davantage le tree-shaking et le code splitting. Par ailleurs, le mot de passe unique 'Admin@2024' pour tous les comptes constitue une vulnerabilite de securite critique qui doit etre corrige rapidement.